Informe técnico: Compiscript MIPS

Índice

1. Documentos del paquete 1
2. Objetivo, entrada y salida 1
3. Matriz de requisitos y evidencia 2
4. Decisiones principales 2
5. Verificación observada 3
6. Problemas detectados y correcciones 3
7. Límites y estado de entrega 3

Construcción de Compiladores · Proyecto Fase 3 · Compiscript.

La fase transforma el código intermedio de tres direcciones de Compiscript en ensamblador MIPS ejecutable en MARS. Conserva el frontend ANTLR, el análisis semántico y las interfaces anteriores; incorpora un backend físico, un runtime de valores y una salida de ensamblador en la CLI y el IDE. La compilación genera código: no interpreta el fuente ni ejecuta el simulador.

1. Documentos del paquete

Documento	Pregunta que responde
Este informe	¿Qué se implementó y qué evidencia respalda los requisitos?
Arquitectura	¿Cómo se organizan el pipeline, la memoria, los registros y el runtime?
Guía de ejecución	¿Cómo instalar lo necesario, compilar, ejecutar y revisar errores?
Referencia TAC a MIPS	¿Cómo se traducen instrucciones y valores concretos?

La transcripción del enunciado conserva las cuatro páginas del PDF recibido. La evidencia de aceptación detalla resultados, fallos corregidos y límites de verificación. Los documentos corresponden a la fase MIPS, no al intérprete SQL que también está presente en el repositorio.

2. Objetivo, entrada y salida

Objetivo: obtener código MIPS funcional a partir de `tac.Program`, incluyendo llamadas, retornos, almacenamiento y las construcciones del lenguaje que ya admitía el frontend.

Entrada: un archivo Compiscript UTF-8 con extensión `.cps`, o su contenido por el IDE/API.

Salida: ensamblador `.asm` o un reporte JSON con `mipsText`, TAC, ámbitos, símbolos y diagnósticos. Si el target falla, `mipsText` queda vacío; puede conservarse el TAC anterior para inspección. Los errores de frontend impiden producir TAC y MIPS.

La ejecución es otro paso: MARS ensambla y simula el archivo generado, con sus propios registros, memoria, syscalls y límites. No se necesita un procesador MIPS físico ni se ejecutan instrucciones MIPS directamente en el procesador del equipo.

3. Matriz de requisitos y evidencia

Requisito del enunciado	Implementación	Evidencia verificable
Secuencias de llamada y retorno — 20 puntos	Frames propios; preservación de `$ra`, `$fp` y `$s`; parámetros por pila; resultado en `$v0`	`TestCallsInMARS`: Fibonacci, recursión mutua, aislamiento de locales, retornos y diez argumentos con llamadas anidadas
Asignación de registros o pila — 20 puntos	`getReg` presta scratch por instrucción; valores con homes canónicos y spilling inmediato	Regresiones de temporales vivos, alias, llamadas recursivas y presión de argumentos; no se afirma un asignador optimizador
Código MIPS funcional y batería — 60 puntos	Traducción directa del IR estructurado y runtime MIPS para valores, referencias y excepciones	Suite dedicada MIPS, 150 escenarios ampliados y demo por CLI ejecutados en MARS real
Fuente `.cps` y reporte de errores	`CompileMIPS`, CLI independiente y endpoint aditivo	Pruebas de CLI/API: éxito, errores, ubicación, ausencia de ensamblador parcial e inmutabilidad del fuente
IDE para escribir y compilar	Editor existente, acción Compile MIPS, pestaña MIPS y descarga `.asm` vigente	41 pruebas web de conexiones/estado/seguridad de texto y API; comprobación visual real pendiente
Arquitectura e instrucciones de uso	Paquete técnico en español y fuentes editables versionadas	Guía, arquitectura, referencia, informe y evidencia enlazados; copias PDF de lectura
Contribuciones individuales de tres integrantes	Deben corresponder al trabajo real del equipo	Control humano: los commits de implementación no certifican por sí solos este requisito

La tabla organiza evidencia: **no adjudica puntos ni garantiza una calificación**. La evaluación requiere funcionamiento en el entorno y fecha de presentación.

4. Decisiones principales

Backend separado, sin romper TAC

`Compile` conserva su contrato previo. `CompileMIPS` lo reutiliza y llama a `mips.Generate` cuando no hay diagnósticos. Las CLI `compiscript` y `compiscript-tac` siguen disponibles; la nueva es `compiscript-mips`. En HTTP, MIPS tiene una ruta propia y comparte el transporte estricto existente, no una ejecución remota de programas.

Pila como base de corrección

Cada variable o temporal tiene una ubicación canónica. Los resultados se guardan inmediatamente; los registros no son la única copia de un valor vivo. Esta estrategia simplifica llamadas, ramas y recursión. El costo es más acceso a memoria y asignaciones de heap, aceptado frente a una optimización prematura.

Los temporales TAC se reciclan y no equivalen a registros MIPS. `getReg` no implementa coloreo de grafos, análisis de vida global ni una política LRU de spills: presta registros scratch y el almacenamiento principal sigue siendo memoria.

Valores etiquetados y entornos retenidos

Las cajas distinguen números, booleanos, strings, null, listas, objetos, funciones y excepciones. Esto permite operaciones dinámicas y reutilización de temporales con tipos diferentes. Los números usan binary64, como el oráculo TAC existente; no se convierten indiscriminadamente a enteros MIPS de 32 bits.

Las capturas de una clausura viven en entornos de heap que sobreviven al retorno del frame. Una nueva entrada de bloque crea celdas nuevas; clausuras hermanas pueden compartir las mismas celdas mutables. Objetos y listas conservan identidad y alias.

Fallos del lenguaje frente a IR inválido

La generación rechaza metadata inconsistente sin devolver ensamblador parcial. En ejecución, división por cero, índices inválidos y otras guardas producen fallos que pueden llegar al handler dinámico más reciente. El desenrollado restaura stack/frame/entornos visibles, pero no revierte efectos ya realizados en celdas compartidas.

5. Verificación observada

El estado del software verificado corresponde al cierre de implementación `115dc4f` y al registro de trabajo posterior, sin cambios funcionales por esta ampliación documental.

- Suite global `go test -json ./... -count=1`: 21 paquetes aprobados; 2379 eventos de pruebas aprobadas, contando padres y subtests.
- Aceptación ampliada: 150 escenarios ejecutados como MIPS real, incluyendo las matrices anteriores de valores/efectos, dinámicos/excepciones, objetos y listas.
- `go vet -a ./...`: sin diagnósticos.
- Web: 41/41 pruebas aprobadas, ninguna omitida.
- MARS smoke y demo pública CLI a MARS: salida exacta y fuente sin cambios.
- Compatibilidad diferencial: Analyze/TAC anteriores conservaron stdout, stderr, salida y respuestas HTTP en los casos comparados.

Las pruebas globales se ejecutaron en una copia aislada, sin red y con cachés/JAR existentes. Los cinco eventos de paquete sin tests no son pruebas omitidas. Dos verificaciones independientes acotadas revisaron la herramienta MARS y la integración compartida CLI/API/IDE. La evidencia identifica el alcance de cada comprobación.

6. Problemas detectados y correcciones

Problema reproducido	Corrección
`%` reducía el cociente a 32 bits	Reducción binaria sobre doubles
Strings UTF-8 se corrompían y NUL cortaba la salida	Bytes con longitud explícita y syscall 15
Cajas nuevas de la misma clausura comparaban distinto	Igualdad por código, entorno y receptor
Un global declarado sin valor se rechazaba antes de poder escribirlo	Sentinel privado de declaración y comprobación separada de lectura
Una clase no usada eludía la validación	Validación incondicional de clases antes de emitir
Una excepción capturada no podía almacenarse en una lista	Descriptor de tipo asociado a la etiqueta 9 ya existente

Los casos que descubrieron las omisiones se conservaron. El primer pase ampliado fue 149/150; después de la corrección y la verificación global, los 150 pasaron.

7. Límites y estado de entrega

El target es MARS 4.5, sin delayed branching activado. Hay memoria finita, heap monotónico sin recolector, límites explícitos de layouts/desplazamientos y una impresión de listas acotada. La representación decimal de MARS puede diferir de Go para números extremos. Los límites completos están en la guía y la arquitectura.

Queda verificar manualmente el navegador real, el entorno del evaluador, acceso a dependencias/CDN y las contribuciones individuales efectivas. No se afirma que esas comprobaciones se hayan realizado automáticamente. La implementación y la documentación se mantienen en una rama local; publicar, abrir un PR y mergear siguen siendo decisiones del equipo.